

NumPy

Veri Madenciliği ve Mak. Öğr. Alg. - Hafta 4

NumPy ile İleri Düzey Dizi Yönetimi

Bu ders, Veri Madenciliği ve Makine Öğrenmesi Algoritmaları çerçevesinde kritik bir araç olan NumPy kütüphanesine odaklanmıştır.

NumPy (Numerical Python), yüksek performanslı çok boyutlu dizi nesneleri ve bu diziler üzerinde işlem yapmak için araçlar sunar.

Üniversite düzeyinde, büyük veri setlerini verimli bir şekilde işlemek için NumPy'in derinlemesine anlaşılması zorunludur.

Bu haftanın temel hedefleri, temel dizi özelliklerinden ileri düzey manipülasyon tekniklerine kadar her şeyi kapsamaktır.



Neden NumPy?

Python'daki standart listeler esnek olsa da, büyük sayısal hesaplamalarda ciddi performans kısıtlamaları vardır.

NumPy'in temel veri yapısı olan 'ndarray', bellekte homojen veri tiplerini tutar, bu da CPU önbellek optimizasyonuna izin verir.

Bu optimizasyon, Veri Madenciliği ve Makine Öğrenmesi algoritmalarının hızını katlanarak artırır.

NumPy, vektörize işlemleri destekleyerek döngü kullanma ihtiyacını minimize eder (C/Fortran hızında işlemler).

NumPy Temelleri: ndarray Nesnesi

NumPy'daki ana nesne, n-boyutlu dizi anlamına gelen 'ndarray'dir. Bu diziler, aynı türden (dtype) öğelerin topluluğudur ve bellekte sabit bir blok halinde depolanır. Bir 'ndarray', dizinin gerçek verisini tutan bir veri bloğundan ve dizinin yapısını tanımlayan meta verilerden (shape, dtype) oluşur. NumPy ile çalışırken bu yapıyı anlamak, bellek verimliliğini yönetmek için önemlidir.

Dizi Oluşturma Yöntemleri (I)

NumPy dizileri genellikle standart Python listelerinden veya tuple'larından oluşturulur.

'np.array()' fonksiyonu en yaygın kullanılan yöntemdir.

Tek boyutlu (vektör) ve iki boyutlu (matris) diziler kolaylıkla oluşturulabilir.

Diziye girdiğimiz elemanların türü, otomatik olarak 'dtype' özelliğini belirler.

Dizi Oluşturma Yöntemleri (II)

'np.zeros()': Belirtilen şekilde ve veri tipinde sıfırlardan oluşan bir dizi oluşturur.
'np.ones()': Belirtilen şekilde birlerden oluşan bir dizi oluşturur.
'np.arange()': Belirtilen aralıkta ve adım büyüklüğünde eşit aralıklı değerler oluşturur (Python'daki 'range' gibi).

Dizi Oluşturma Yöntemleri (III)

'np.linspace()': Belirtilen başlangıç ve bitiş değerleri arasında eşit aralıklı belirtilen sayıda örnek oluşturur (veri görselleştirmede sık kullanılır).
'np.eye()' veya 'np.identity()': Birim matris (köşegenleri 1, diğerleri 0) oluşturur.
'np.random.rand()' veya 'np.random.randn()': Makine öğrenmesinde model başlatma ve simülasyonlar için rastgele diziler oluşturur.



NumPy Dizi İncelemesi: shape

'shape', bir NumPy dizisinin boyutlarını bir tuple olarak döndürür. Örneğin, '(4, 3)' şekli, dizinin 4 satır ve 3 sütundan oluştuğu anlamına gelir. Makine öğrenmesinde, girdi verilerinin ve model çıktıların uyumlu şekillere sahip olması kritiktir. Tek boyutlu bir dizi (vektör) için 'shape' çıktısı '(N,)' şeklinde olur, burada N eleman sayısıdır.



NumPy Dizi İncelemesi: ndim

'ndim' (number of dimensions), dizinin eksen sayısını (boyutluluğunu) gösterir. Tek boyutlu dizilerde 1, matrislerde 2, küp veya tenzörlerde 3 veya daha fazla olabilir.

Derin öğrenmede 4D (Görüntü verileri için: (örnek, yükseklik, genişlik, kanal)) veya 5D (Video verileri) dizilerle karşılaşılabilir.

'ndim' özelliği, algoritmanın veri yapısını doğru bir şekilde anlaması için temeldir.



NumPy Dizi İncelemesi: size

'size', dizideki toplam eleman sayısını verir.

Bu, dizinin bellekte kapladığı alana orantılıdır ve büyük veri setlerinin büyüklüğünü hızlıca kontrol etmek için kullanılır.

'size', tüm boyutların çarpımına eşittir ('shape' tuple'ındaki tüm elemanların çarpımı).



NumPy Dizi İncelemesi: dtype

'dtype' (data type), dizinin içerdiği elemanların tipini belirtir (örneğin: int64, float32, bool).

NumPy, Python'ın yerleşik tiplerine göre daha spesifik ve optimize edilmiş tiplere sahiptir (int32, int64, float32, float64).

Veri tiplerinin doğru seçilmesi, bellek kullanımını ve hesaplama hızını doğrudan etkiler.

Özellikle makine öğrenmesinde, eğitim hızını artırmak için genellikle daha düşük hassasiyetli (float32) tipler tercih edilir.

NumPy Dizi İncelemesi: itemsize ve nbytes

'itemsize', dizideki tek bir elemanın bayt cinsinden kapladığı alanı gösterir. 'nbytes', dizinin toplam bellekte kapladığı alanı gösterir ('size * itemsize"a eşittir).

Büyük veri setleriyle çalışırken, bellek verimliliğini izlemek ve optimizasyon yapmak için bu özellikler hayati önem taşır.

Örneğin, 'int64' 8 bayt, 'float32' ise 4 bayt yer kaplar.

Uygulama: Temel Özellik Sorgulama

Basit bir 2x3 matrisi oluşturalım: `a = np.array([, ])`:

- `'a.shape'` çıktısı: `'(2, 3)'`. (2 satır, 3 sütun)
- `'a.ndim'` çıktısı: `'2'`. (iki boyutlu)
- `'a.size'` çıktısı: `'6'`. (Toplam 6 eleman)
- `'a.dtype'` çıktısı: `'dtype('int64)'`. (Elemanlar 64-bit tam sayı)

Veri Tipi Dönüştürme (Casting)

NumPy dizilerinde veri tipleri, 'astype()' metodu kullanılarak dönüştürülebilir. Örneğin, 'int' tipindeki bir diziyi 'float' tipine çevirmek veya bellekten tasarruf etmek için 'float64'ten 'float32'ye geçiş yapmak mümkündür. Tip dönüştürme, özellikle veri hazırlığı aşamasında ve farklı kütüphanelerle entegrasyon sağlanırken önemlidir.

İndekslemeye Giriş

İndeksleme, bir dizinin tek tek elemanlarına erişmek için kullanılan temel yöntemdir.

NumPy dizilerinin indeks numaraları her zaman `**0` (sıfır) ile başlar**.

Tek boyutlu dizilerde erişim, Python listelerindeki gibidir: `'dizi[i]'`.

Çok boyutlu dizilerde ise her eksen için ayrı indeks belirtilir: `'dizi[satır, sütun]'` veya `'dizi'`.

Tek Boyutlu İndeksleme

Tek boyutlu bir vektörde ($\text{ndim}=1$), pozitif indeksleme soldan (0'dan başlayarak) ilerler.

Negatif indeksleme, dizinin sonundan başlayarak elemanlara erişim sağlar (-1 son elemandır).



İki Boyutlu İndeksleme

2D dizilerde (matrisler), erişim '[satır_indeksi, sütun_indeksi]' formatında yapılır.

Bu yapı, matris işlemlerinde belirli bir hücredeki değere erişimi veya o değeri değiştirmeyi sağlar.

İlk indeks, dıştaki boyut (satır), ikinci indeks ise içteki boyut (sütun) ile ilişkilidir.

Dilimleme (Slicing) Kavramı

Dilimleme (Slicing), NumPy dizisinden birden çok elemanı veya bir alt diziyi seçme işlemidir.

Standart kullanım biçimi: '[başlangıç:bitiş:adım]' şeklindedir.

Bitiş indeksi ****dahil değildir**** (exclusive) – bu, Python'ın genel dilimleme kuralıdır.

Dilimleme, büyük veri setlerinden analiz için küçük parçalar çıkarmada çok etkilidir.

Temel Dilimleme Kuralları

Başlangıç (Start): Seçime dahil edilecek ilk indeks.

Bitiş (Stop): Seçime dahil edilmeyecek, dilimin bittiği indeks.

Adım (Step): Kaç eleman atlanacağını belirler (Varsayılan: 1).

Eğer başlangıç veya bitiş belirtilmezse, varsayılan olarak dizinin başına veya sonuna kadar gider.

'[:]' kullanımı, dizinin tam bir kopyasını (view olarak) döndürür.

Çok Boyutlu Dilimleme (2D)

2D dizilerde dilimleme, her boyut için ayrı ayrı yapılabilir: '[satır_dilimi, sütun_dilimi]'.

Tüm satırları seçmek için sadece kolon dilimi belirtilir: 'dizi[:, 1:3]' (Tüm satırların 1. ve 2. sütunları).

Belirli satırları seçmek için sadece satır dilimi belirtilir: 'dizi[0:2, :]' (0. ve 1. satırların tüm sütunları).

Bu, matrisin alt bölgelerini kolayca seçmemizi sağlar.

Adımlı Dilimleme (Step Slicing)

Adım parametresi, seçilen aralıktaki hangi elemanların alınacağını belirler. Örneğin, 'dizi[::2]' her iki elemandan birini seçerek dizinin yarısını alır. Negatif adım ('[::-1]') kullanmak, diziyi ters çevirmek için pratik bir yöntemdir. Bu teknik, büyük veri setlerinde hızlı örnekleme (sampling) yapmak için kullanılabilir.

İndeksleme ve Dilimleme Farkı

Temel indeksleme (tek bir elemana erişim), bir Python nesnesi (sayı) döndürür.

Dilimleme ise her zaman bir NumPy ****görünümü (view)**** döndürür.

Önemli not: Dilimleme ile elde edilen görünüm üzerinde yapılan değişiklikler, orijinal diziyi de etkiler.

Bu, bellek kopyalama maliyetini düşürmek için NumPy'ın kullandığı kritik bir optimizasyondur.



Boolean (Maske) indeksleme

Boolean indeksleme, dizinin elemanları üzerinde bir koşul uygulayarak 'True'/'False' değerlerinden oluşan bir maske oluşturmaktır. Sadece maskede 'True' olan pozisyonlardaki elemanlar seçilir. Bu yöntem, veri temizliği, aykırı değer tespiti ve filtreleme işlemlerinde son derece güçlüdür.

Fancy indeksleme

Fancy indeksleme, elemanların pozisyonlarını belirten tam sayı dizileri veya listeleri kullanarak rastgele seçim yapmaktır.

Örneğin, 'dizi[]' sadece 0., 4. ve 1. indekslerdeki elemanları döndürür.

Fancy indeksleme, dilimlemeden farklı olarak, her zaman orijinal dizinin bir ****kopyasını**** (copy) döndürür.

Bu, özellikle makine öğrenmesinde eğitim ve test verisi örneklemesi için kullanışlıdır.



İleri Düzey Manipülasyonlar

Temel indeksleme ve dilimleme becerilerimizi geliştirdikten sonra, dizilerin yapısını değiştirmeye odaklanacağız.

Bu, verinin analize veya model girişine uygun hale getirilmesi için kritik bir adımdır.

İleri manipülasyonlar şunları içerir: Yeniden şekillendirme ('reshape'), birleştirme ('concatenate'), ayırma ('split') ve sıralama ('sort').

Yeniden Şekillendirme (Reshape)

'reshape()' metodu, dizinin toplam eleman sayısı ('size') değişmemek koşuluyla boyutlarını yeniden düzenler.

Örneğin, 12 elemanlı bir vektörü 4x3 veya 2x6 matrisine dönüştürebiliriz. Çok boyutlu diziyi tek boyutlu hale getirmek için 'flatten()' veya 'ravel()' metotları da kullanılabilir.

'reshape(-1)' kullanımı, NumPy'in diğer boyutu otomatik olarak hesaplamasını sağlar (sık kullanılan bir tekniktir).



Boyut Ekleme ve Çıkarma

'np.newaxis': Bir dizinin belirli bir pozisyonuna yeni bir boyut (eksen) eklemek için kullanılır.

Örneğin, makine öğrenmesi modellerine tek bir görüntüyü verirken, '(Yükseklik, Genişlik)' olan şekli '(1, Yükseklik, Genişlik)' şekline getirmek gerekebilir.

'squeeze()': Şekli 1 olan tüm boyutları diziden kaldırır (örneğin, 3x1 matrisini 3 elemanlı vektöre dönüştürür).

Dizileri Birleştirme (Concatenation)

'np.concatenate()', birden fazla diziyi tek bir dizide birleştirmek için kullanılır. Bu işlemde temel kural, birleştirilecek dizilerin, birleştirme eksenini (axis) hariç, tüm boyutlarının aynı olmasıdır.

'axis=0' (varsayılan): Satırlar boyunca (dikey) birleştirme.

'axis=1': Sütunlar boyunca (yatay) birleştirme (2D ve üzeri için).



Dikey Birleştirme

'np.vstack()' (vertical stack), dizileri satırlar boyunca (dikey olarak) birleştirmenin kısaltılmış yoludur.

Bu, 'np.concatenate(..., axis=0)' ile eşdeğerdir.

Özellikle makine öğrenmesinde, eğitim setine yeni veri örnekleri (satırlar) eklemek için sıkça kullanılır.

Yatay Birleştirme

'np.hstack()' (horizontal stack), dizileri sütunlar boyunca (yatay olarak) birleştirmenin kısaltılmış yoludur.

Bu, 'np.concatenate(..., axis=1)' ile eşdeğerdir.

Farklı özellik kümelerini veya etiket sütunlarını mevcut veri setine (matrise) eklemek için kullanılır.



Derinlik Boyunca Birleştirme

'np.dstack()' (depth stack), dizileri üçüncü bir eksen (derinlik) boyunca birleştirir.

Bu, genellikle 3D veriler (örneğin RGB kanalları) veya tenzörlerle çalışırken kullanılır.

Genel yığınlama fonksiyonu 'np.stack()', yeni bir eksen oluşturarak birleştirme yapar (concatenate'den farkı budur).



Dizileri Ayırma (Splitting)

'np.split()', tek bir diziye birden fazla küçük parçaya bölmek için kullanılır. Ayırma işlemi için dizinin hangi eksen (axis) boyunca bölüneceği ve kaç eşit parçaya ayrılacağı belirtilmelidir. Ayırma işlemi, makine öğrenmesi veri setlerini eğitim, doğrulama ve test kümelerine ayırmak için önemlidir.



Yatay ve Dikey Ayırma

'np.vsplit()' (vertical split), diziyi satırlar boyunca (dikey olarak) böler.
'np.hsplit()' (horizontal split), diziyi sütunlar boyunca (yatay olarak) böler.
Bu kolaylaştırıcı fonksiyonlar, 'np.split()'ın sırasıyla 'axis=0' ve 'axis=1' parametreleriyle çağrılmasına eşdeğerdir.

Dizileri Sıralama (Sorting)

'np.sort()', bir dizinin bir kopyasını sıralanmış olarak döndürür. Orijinal dizi değişmez.

Dizinin yerinde (in-place) sıralanması için, dizi objesinin kendi metodu olan 'dizi.sort()' kullanılır.

Sıralama, veri analizi ve istatistiksel işlemlerden önce verinin düzenlenmesini sağlar.



Eksen Boyunca Sıralama

Çok boyutlu dizilerde, sıralama işleminin hangi eksen boyunca yapılacağı 'axis' parametresi ile belirlenir.

'axis=0': Her sütunun içindeki değerler ayrı ayrı sıralanır.

'axis=1': Her satırın içindeki değerler ayrı ayrı sıralanır.

'axis=None': Dizi tamamen tek boyutlu hale getirilip sıralanır.



Argüman Sıralama (argsort)

'np.argsort()', doğrudan değerleri sıralamak yerine, sıralı dizideki elemanların orijinal indekslerini döndürür.

Bu, bir veri setindeki bir sütunu sıralarken, diğer ilgili sütunların da aynı şekilde yeniden düzenlenmesi gerektiğinde hayati öneme sahiptir.

Fancy indeksleme ile birleştirilerek veri setinin sırası bozulmadan sıralı hale getirilmesi sağlanır.



Broadcasting Kavramına Giriş

Broadcasting, farklı şekillere sahip diziler arasında aritmetik işlemleri mümkün kılan bir dizi kuralıdır (Örn: Matrise tek bir sayı ekleme). NumPy, daha küçük diziyi daha büyük dizinin şekliyle eşleştirmeye çalışır. Bu mekanizma, veri biliminde sabit bir değeri tüm diziye ekleme (merkezeleme) gibi işlemleri hızlı ve bellek açısından verimli yapar.



Broadcasting Kuralları (I)

Kural 1: Eğer dizilerin boyutları (ndim) farklıysa, daha küçük boyuta sahip diziye sol taraftan 1 eklenir.

Kural 2: Boyutlar karşılaştırılırken, her iki dizideki boyutlar eşit olmalı veya boyutlardan biri 1 olmalıdır.

Bu kurallar, NumPy'in hangi işlemlerin yasal olduğunu belirlemesini sağlar.

Broadcasting Kuralları (II)

Eğer iki dizinin boyutları ne eşitse ne de biri 1 ise (örneğin (3, 4) ile (5, 4) çarpımı), Broadcasting mümkün olmaz ve hata ('ValueError') oluşur. Broadcasting, diziyi manuel olarak kopyalamaktan veya genişletmekten (tiling) daha verimlidir. Veri normalizasyonu (ortalama çıkarma ve standart sapmaya bölme) genellikle Broadcasting kullanılarak gerçekleştirilir.

NumPy ve İstatistiksel Fonksiyonlar

NumPy, hızlı ve optimize edilmiş istatistiksel hesaplamalar sunar.

- 'np.mean()': Dizinin ortalamasını hesaplar.
- 'np.median()': Dizinin medyanını (ortanca) hesaplar.
- 'np.std()': Standart sapmayı hesaplar (Veri yayılımını ölçer).

Eksen Boyunca İstatistik

İstatistiksel fonksiyonlar da 'axis' parametresini kullanır.

'np.mean(dizi, axis=0)': Her sütunun ortalamasını hesaplar (özellik bazında ortalama).

'np.mean(dizi, axis=1)': Her satırın ortalamasını hesaplar (örnek bazında ortalama).



Matematiksel İşlemler ve Evrensel Fonksiyonlar (UFuncs)

NumPy, diziler üzerinde eleman bazında hızlı işlemler yapmak için UFuncs (Universal Functions) kullanır.
Örnekler: `'np.sqrt()'`, `'np.exp()'`, `'np.log()'`, trigonometrik fonksiyonlar.
UFuncs, veriyi hızlı bir şekilde dönüştürmemizi sağlar ve Python döngülerinden kaçınarak performansı artırır.

Dizideki Değerleri Kısıtlama (Clipping)

'np.clip(dizi, min, max)' fonksiyonu, dizideki değerleri belirlenen minimum ve maksimum değerler arasında kısıtlar.

Eğer bir eleman min'den küçükse min değerini alır, max'tan büyükse max değerini alır.

Bu, özellikle makine öğrenmesinde gradyan patlamasını önlemek veya görüntü işleme alanında piksel değerlerini belirli bir aralıkta tutmak için önemlidir.



Lineer Cebir Temelleri (I)

NumPy, matris ve vektör işlemlerini kolaylaştırır.
İki matrisi çarpmak için '@' operatörü veya 'np.dot()' fonksiyonu kullanılır.
Önemli: NumPy'da '*' operatörü eleman bazlı çarpma (element-wise multiplication) yapar, matris çarpımı değil.



Lineer Cebir Temelleri (II)

Matrisin transpozu ('dizi.T' veya 'np.transpose()'), matrisin satırlarını sütun, sütunlarını satır yapar.

'np.linalg.inv()': Matrisin tersini hesaplar (eğitim ve optimizasyon algoritmalarında kullanılır).



Görünüm vs. Kopya (View vs. Copy) Detaylı İnceleme

NumPy, performansı artırmak için mümkün olduğunca ****görünüm**** (view) döndürmeyi tercih eder.

Bir görünüm, orijinal verinin bellekteki aynı bloğunu işaret eder. Görünümdeki değişiklikler orijinal diziyi etkiler.

****Kopya**** (copy) ise verinin tamamen yeni bir bellek bloğudur. Kopyadaki değişiklikler orijinali etkilemez.

Hangi işlemin görünüm, hangi işlemin kopya döndürdüğünü bilmek (özellikle dilimleme ve fancy indeksleme) kritik hataları önler.



NumPy Dosya İşlemleri

'np.save()': Bir diziyi sıkıştırılmamış ikili formatta (.npy uzantısı) kaydeder.

'np.load()': Kaydedilmiş .npy dosyasını belleğe geri yükler.

'np.savetxt()' ve 'np.loadtxt()': Veriyi metin formatında (CSV benzeri) kaydetmek ve yüklemek için kullanılır.

NumPy'in yerel formatı (.npy), veri tiplerini ve şekilleri koruduğu için önerilen yöntemdir.



Hata Ayıklama ve İpuçları (Debugging)

Shape Hataları: En sık karşılaşılan hata, işlem yapılan dizilerin şekillerinin Broadcasting veya Matris Çarpımı kurallarına uymamasıdır (Örn: 'ValueError: operands could not be broadcast together').

dtype Hataları: Farklı veri tiplerini birleştirmeye çalışmak veya float tipinde bir diziye tam sayı atamak.

View/Copy Hataları: Görünüm üzerinde değişiklik yapmaya çalışırken orijinal veriyi yanlışlıkla değiştirmek.



NumPy Performans Optimizasyonu

NumPy'i kullanmanın temel amacı hızdır.

Her zaman yerleşik NumPy fonksiyonlarını kullanın (Vektörizasyon) ve yavaş Python döngülerinden kaçının.

Bellek erişimini optimize etmek için dizileri sıralı bellekte tutmaya dikkat edin (C-sırası varsayılanıdır).



NumPy ve Diğer Kütüphanelerle Etkileşim

Pandas: DataFrame'ler, altında NumPy dizilerini kullanır. Veri temizleme ve hazırlık aşamasında birlikte kullanılırlar.

SciPy: NumPy'ın sağladığı temel dizi yapısı üzerine inşa edilmiş bilimsel hesaplama kütüphanesidir (Örn: optimizasyon, sinyal işleme).

Matplotlib: Görselleştirme kütüphanesi, grafik çizmek için girdi olarak NumPy dizilerini kabul eder.

Örnek Çalışma: Görüntü İşleme Temelleri

Bir renkli görüntü, NumPy'da genellikle (Yükseklik, Genişlik, Renk Kanalları) şeklinde 3D bir dizi olarak temsil edilir.

NumPy'in indeksleme ve dilimleme yetenekleri, görüntünün belirli bir bölgesini seçmeyi, renk kanallarını ayırmayı (Kırmızı, Yeşil, Mavi) veya parlaklık ayarı yapmayı sağlar.

Bu, 'ndim' ve 'shape' özelliklerinin somut bir uygulamasıdır.

Örnek Çalışma: Matris Çözümleri

NumPy'in lineer cebir modülü ('np.linalg'), doğrusal denklem sistemlerini çözmek için kullanılabilir.

Örneğin, ' $Ax = B$ ' denklemini çözmek için 'np.linalg.solve(A, B)' kullanılır.

Bu, birçok makine öğrenmesi algoritmasının (örneğin Regresyon) temelinde yatan matematiksel operasyonları hızlandırır.



Dönem Sonu Projesi Girişi (NumPy)

Öğrencilerin bu hafta öğrendiği ileri düzey NumPy manipülasyonlarını kullanarak gerçekleştirebileceği potansiyel projeler:

1. Görüntüye filtre uygulama (Siyah-beyaz dönüştürme, kenar tespiti).
2. Bir matrisin Özet Değer Ayrışımını (SVD) kullanarak temel bileşen analizi (PCA) uygulaması.
3. Büyük bir veri kümesini (CSV/Txt) okuma, temizleme ve normalizasyon süreçlerini NumPy ile tamamen gerçekleştirme.

Özet: Temel Dizi Özellikleri

'shape': Boyutlar (Satır x Sütun, vb.).

'ndim': Boyut sayısı (Eksen sayısı).

'size': Toplam eleman sayısı.

'dtype': Elemanların veri tipi (int64, float32).

Bu özellikler, dizinin yapısını tanımlayan meta verilerdir ve manipülasyon işlemlerinden önce mutlaka kontrol edilmelidir.

Özet: Erişim ve Seçim Teknikleri

İndeksleme: Tek elemana veya belirli bir konuma erişim (0'dan başlar).

Dilimleme: Alt diziler seçme ('[başlangıç:bitiş:adım]'). Çoğunlukla ****görünüm**** döndürür.

Fancy İndeksleme: Rastgele indeks listesiyle eleman seçimi (Her zaman ****kopya**** döndürür).

Boolean İndeksleme: Koşul bazlı filtreleme için kullanılır.

Özet: Manipülasyon Metotları

'reshape()': Eleman sayısını değiştirmeden dizinin boyutlarını yeniden ayarlar.
'concatenate()' / 'stack()': Dizileri belirli bir eksen boyunca birleştirir (dikey, yatay).
'split()' / 'vsplit()' / 'hsplit()': Bir diziyi birden fazla parçaya ayırır.
'sort()': Dizideki elemanları sıralar.

Gelecek Haftanın Konusu

Gelecek hafta, NumPy'in hesaplama gücünü tam olarak nasıl kullanacağımızı inceleyeceğiz.

Vektörizasyon tekniklerinin derinlemesine anlaşılması.

UFuncs'un (Evrensel Fonksiyonlar) ileri düzey kullanımı.

Numpy ve C entegrasyonuna kısa bir giriş.



Soru ve Cevap

Bu hafta ele aldığımız konularla ilgili sorularınızı yanıtlamaktan memnuniyet duyarım.

Konu: shape, ndim, size, dtype, indeksleme, dilimleme, reshape, concatenate, split ve sort.

Hedef Kitle: Üniversite Düzeyi Veri Bilimi Öğrencileri.



Numpy

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

